

Solana Dip Bot

V2 Architecture & Development Plan

Stack	Python 3.11 · asyncio · aiohttp · aiosqlite · pydantic-settings
Data Sources	Helius RPC · Helius WebSocket · DexScreener (free) · Solana RPC · Pump.fun
Storage	SQLite (local) — upgrade path to PostgreSQL documented
Deployment	Railway (Docker) or bare VPS · ~40–80 Helius credits/hr
BirdEye	ZERO dependency — fully replaced with local snapshot storage
Version	v1.0.0 · Stage 1 complete · Stages 2–5 begin on approval

What V2 Fixes

BirdEye removed	All ATH history now stored locally from first-seen. Zero paid candle APIs.
No rate-limit risk	DexScreener: 1 batched call per 45s. Helius WS replaces RPC polling.
Clean layers	clients / detectors / services / storage / telegram — no god files.
Deterministic scores	Every component labeled: computed vs heuristic. No fake precision.
State machine	Explicit transitions, logged reasons, no overlapping filter drift.
Cheap infra	SQLite + ~40–80 Helius credits/hr. Well within free tier.

Folder Structure

Complete file layout with layer annotations

directory	.py	.sql	.yaml/.yml	other	comment
solana_dip_bot_v2/					
■					
■■■■ app/					
■	■■■■ main.py				# entry point, wires
■	■■■■ runner.py				# async loop, task s
■	■■■■ settings.py				# pydantic-settings,
■					
■■■■ clients/ # raw API wrappers onl					
■	■■■■ helius_rpc.py				# getTokenAccounts,
■	■■■■ helius_ws.py				# WebSocket: account
■	■■■■ dexscreener.py				# batched lookup, ra
■	■■■■ solana_rpc.py				# public RPC fallback
■	■■■■ pumpfun.py				# migration feed po
■					
■■■■ core/ # pure logic, no I/O,					
■	■■■■ enums.py				# TokenStatus, Alert
■	■■■■ models.py				# TokenState, Snapsh
■	■■■■ state_machine.py				# transition guards
■	■■■■ scoring.py				# 6-component 0-100
■	■■■■ thresholds.py				# all numeric consta
■					
■■■■ detectors/ # domain detectors, pu					
■	■■■■ migration_detector.py				
■	■■■■ ath_tracker.py				# ATH from loca
■	■■■■ retracement_detector.py				
■	■■■■ flow_detector.py				# pool reserve
■	■■■■ holder_risk_detector.py				
■	■■■■ dev_risk_detector.py				
■	■■■■ launch_risk_detector.py				
■	■■■■ repeat_pump_detector.py				
■					
■■■■ services/ # orchestration, drive					
■	■■■■ token_service.py				# token lifecycle
■	■■■■ market_service.py				# DexScreener ba
■	■■■■ snapshot_service.py				# persists all s
■	■■■■ risk_service.py				# assembles full
■	■■■■ alert_service.py				# tier eval, cod
■	■■■■ cache_service.py				# in-memory LRU
■					
■■■■ storage/					
■					
■	■■■■ db.py				# aiosqlite engine, pool
■	■■■■ migrations/				
■	■■■■ 001_initial.sql				# full schema
■	■■■■ repositories/				
■	■■■■ token_repo.py				
■	■■■■ snapshot_repo.py				
■	■■■■ pool_repo.py				
■	■■■■ risk_repo.py				
■	■■■■ alert_repo.py				
■					
■■■■ integrations/					
■	■■■■ telegram/				
■	■■■■ sender.py				# async bot sender
■	■■■■ formatter.py				# builds message from Scor
■	■■■■ templates.py				# per-tier message templat
■					
■■■■ utils/					
■	■■■■ retry.py				# exponential backoff decora
■	■■■■ log.py				# structlog setup
■	■■■■ math_utils.py				# pct_change, clamp, weighte
■					
■■■■ tests/					
■	■■■■ test_state_machine.py				
■	■■■■ test_scoring.py				
■	■■■■ test_detectors.py				
■	■■■■ test_alert_service.py				
■					
■■■■ scripts/					
■	■■■■ migrate_db.py				
■	■■■■ backfill_snapshots.py				
■	■■■■ health_check.py				
■					
■■■■ config/					
■	■■■■ default.yaml				
■					
■	■■■■ .env.example				
■	■■■■ requirements.txt				
■	■■■■ Dockerfile				
■	■■■■ docker-compose.yml				
■	■■■■ README.md				

API Budget

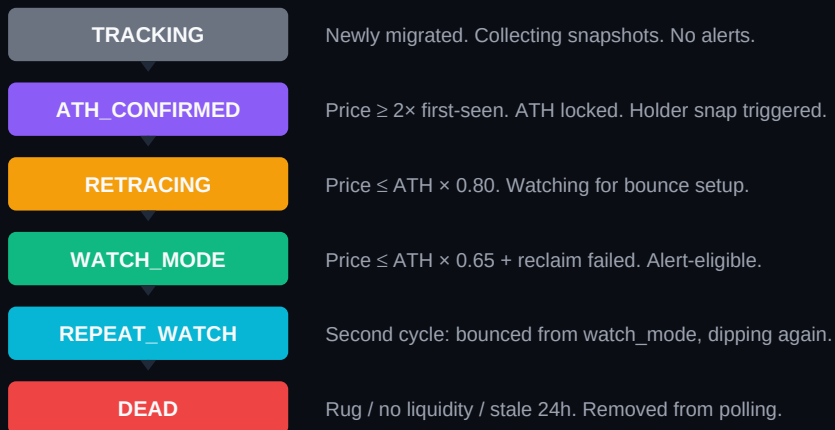
Steady-state credit usage · 30 actively tracked tokens

Source	Call Type	Frequency	Credits/hr	Notes
DexScreener	Batch token lookup (≤ 30 addr)	1 req / 45s	0 — free	~1–2 req/min · limit 80/min
Helius WS	accountSubscribe (pool acct)	Subscribe once	0	Push-based, no polling
Helius RPC	getTokenAccounts (holders)	1 per token / 10min	~1/call	Only ath_confirmed+ state
Helius RPC	getAccountInfo (dev wallet)	1 per token / 15min	~1/call	Top 5 tokens only
Helius RPC	getAssetsByOwner (migration)	On event only	~2/call	WS-triggered, not polled
Solana RPC	getAccountInfo (pool res.)	WS push covered	0	Fallback only
Pump.fun	Migration feed (public)	Poll 30s	0	No auth required
TOTAL (30 tkns)		Steady state	~40–80/hr	< 50k/day free tier ✓

■ **Hard cap: max 50 tracked tokens. Dead tokens pruned immediately. Holder snapshots only run for ath_confirmed / watch_mode states.**

Token State Machine

All transitions are logged to lifecycle_events



Transition Guards

TRACKING → ATH_CONFIRMED	<code>price_usd ≥ first_seen_price × 2.0 (ATH_MIN_MULTIPLE)</code>
ATH_CONFIRMED → RETRACING	<code>price_usd ≤ ath_price × 0.80 (RETRACE_START_PCT 20%)</code>
RETRACING → WATCH_MODE	<code>price_usd ≤ ath_price × 0.65 AND reclaim_failed_window elapsed (1800s default)</code>
WATCH_MODE → REPEAT_WATCH	<code>price bounced ≥ 30% from local_low THEN drops again</code>
ANY → DEAD	<code>liquidity_usd < 5000 OR pair inactive > 6h OR rug signal</code>

Scoring Engine

Total score = weighted sum of 6 components · range 0–100

Market Quality

25%

- liquidity_usd tiers → 0–40 pts
- liq / mcap ratio (healthy 5–15%) → 0–25 pts
- buy/sell balance from 5m + 1h windows → 0–20 pts
- volume consistency (5m / 1h / 24h spread) → 0–15 pts

Holder Health

20%

- top10_pct ≤ 20% = healthy · ≥ 50% = danger → 0–50 pts
- top20_pct ≤ 35% = healthy · ≥ 65% = danger → 0–30 pts
- holder_count tiers (≥500 good · ≥2000 excellent) → 0–20 pts

Dev Behavior

15%

- sold_pct 0% = 100 pts · ≥80% sold = 0 pts (linear scale)
- dev_status: HOLDING=100 · PARTIAL=60 · DUMPED=20 · EXIT=0
- computed from first-observed vs current balance (not BirdEye)

Flow Strength

15%

- pool reserve 15m delta direction (+/–) → directional score
- pool reserve 1h cumulative flow → trend confirmation
- FlowSignal: STRONG_BUY=100 · BUY=75 · NEUTRAL=50 · SELL=25 · STRONG_SELL=0

Launch Quality

10%

- HEURISTIC — labeled in score_breakdown_json
- early buyer count in first 10 slots → breadth score
- if early slot data unavailable → default 50 (neutral, not penalized)

Recovery Structure

15%

- retracement_depth 35–50% = ideal dip zone → max score
- local_low_stabilization: price holding above low N minutes → +pts
- reclaim_failure_confirmed: waited full window before watch_mode → +pts
- rebound_quality: early price action off local low direction → +pts

Score → Alert Tier Mapping

Tier	Min Score	Min Liquidity	Required State	Cooldown
Tier 1 — STRONG	≥ 75	\$30k+	watch_mode / repeat_watch	4h
Tier 2 — MODERATE	≥ 60	\$15k+	watch_mode / repeat_watch	6h
Tier 3 — SPECULATIVE	≥ 45	\$8k+	watch_mode	8h
Repeat Pump	≥ 65	\$20k+	repeat_watch	2h

Database Schema

SQLite · repository pattern · no raw SQL in business logic

<code>tokens</code>	Core token lifecycle record — status, ATH, local low, deployer wallet
<code>price_snapshots</code>	DexScreener data per poll: price, mcap, liquidity, volume windows, buys/sells
<code>pool_snapshots</code>	Pool reserve reads via Helius WS: base/quote reserves, derived flow signals
<code>holder_snapshots</code>	Top holder snapshot: holder_count, top10_pct, top20_pct, top_holder_json
<code>dev_wallet_snapshots</code>	Deployer token balance over time: initial, current, sold_pct, DevStatus
<code>risk_snapshots</code>	Full score snapshot: all 6 components + breakdown JSON + alert_tier
<code>alerts</code>	Alert history: tier, fired_at, price/mcap at time, Telegram message_id
<code>lifecycle_events</code>	Every state transition: from, to, reason, metadata JSON, timestamp

BirdEye Removal Map

Every V1 BirdEye dependency and its V2 replacement

V1 BirdEye Usage	V2 Replacement	Location
Historical candle fetch (ATH detect)	Local <code>price_snapshots</code> from first-seen <code>ath_tracker.py</code> computes ATH forward	<code>detectors/ath_tracker.py</code>
Lifetime fee organic signal	Removed. <code>MarketQuality</code> uses DexScreener volume + buy/sell ratio instead	<code>core/scoring.py</code>
<code>BIRDEYE_API_KEY</code> in config	Removed from <code>settings.py</code> entirely. Env var does not exist in V2	<code>app/settings.py</code>
BirdEye client module	File does not exist. <code>clients/</code> has no <code>birdeye.py</code>	<code>clients/</code> (absent)
BirdEye links in Telegram alerts	DexScreener pair link + Solscan token link used instead	<code>telegram/templates.py</code>
Wallet age from BirdEye	Not computed. <code>LaunchQuality</code> uses on-chain first-tx slot heuristic	<code>detectors/launch_risk_detector.py</code>

Development Stages

Approval-gated · versioned · zero code before Stage 1 sign-off

Stage 1 Architecture & Approval NOW

- This document. Full architecture review.
- DB schema, state machine rules, scoring weights all locked.
- → Approval gate: no code written until you sign off.

Stage 2 Foundation ~2-3 hrs

- app/settings.py · storage/db.py · 001_initial.sql
- All 5 client files · All 5 repositories · utils/
- core/enums.py · core/models.py · core/thresholds.py
- Git tag: v1.0.0

Stage 3 Core Logic ~3-4 hrs

- core/state_machine.py + core/scoring.py
- All 8 detector files
- services/snapshot_service.py · services/risk_service.py
- Git tag: v1.1.0

Stage 4 Orchestration ~2-3 hrs

- `services/token_service.py` · `market_service.py` · `alert_service.py` · `cache_service.py`
- `integrations/telegram/` (sender + formatter + templates)
- `app/runner.py` · `app/main.py`
- Git tag: **v1.2.0**

Stage 5 Tests & Finalization ~1-2 hrs

- tests/ — state machine, scoring, detectors, alert service
- scripts/ — migrate_db, backfill_snapshots, health_check
- config/default.yaml · .env.example · Dockerfile · docker-compose.yml
- Git tag: v1.3.0 (release candidate)

Hard Block Rules — No Alert Fires If:

.env.example

```
# ■■ Helius ████████████████████████████████████████████████████████████  
HELIUS_API_KEY=your_helius_api_key_here HELIUS_RPC_URL=https://mainnet.helius-rpc.com/?api-key=...  
HELIUS_WS_URL=wss://mainnet.helius-rpc.com/?api-key=... # ■■ Telegram
```

```
TELEGRAM_CHAT_ID=your_chat_id_here # Bot behavior
MAX_TRACKED_TOKENS=50 SNAPSHOT_INTERVAL_SEC=45 HOLDER_SNAPSHOT_INTERVAL_SEC=600
DEV_SNAPSHOT_INTERVAL_SEC=900 DB_PATH=./data/dipbot.db # Score thresholds
ATH_MIN_MULTIPLE=2.0 RETRACE_START_PCT=0.20
WATCH_FLOOR_PCT=0.35 RECLAIM_WINDOW_SEC=1800 MIN_LIQUIDITY_USD=5000
```